

# Trippy

---

## Trippy (*student proposal*)

---

**Assessment:** ♦ Fit with adjustments · **Category:** Social Media · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

**Proposal (as submitted):** A social travel platform that turns authentic user journeys into actionable itineraries. Instead of static POI databases or scraped reviews, Trippy builds its spatial dataset from verified traveller activity: as users map destinations, upload photos and categorize locations, a proprietary C++ engine infers dwell time, pacing, seasonality and spot popularity. Named technical features: an **intent-based itinerary search engine** (ranks complete multi-day itineraries against constraints, prioritising trips completed by social connections); a **self-evolving spatial metadata pipeline** (infers place metadata from minimal inputs); **crowd-validated entity resolution** (human-in-the-loop consensus for duplicate/offset entries via proximity prompts); and **on-device instant itinerary optimization** (a C++ engine reorders stops/day-clusters to a new user's pace/schedule/seasonality without network latency, streaming improved itineraries live). Custom tool: a C++ desktop editor sharing the engine — add places, seed starter itineraries, retune recommender weights and watch rankings re-sort, plus audio config; data-driven so behaviour changes without recompiling. Design features: collaborative trip planning (create/discover/plan-with-friends/shared itinerary); friend-sourced discovery (recommendations, save/organise, activity voting); travel details & tracking (document storage, day-by-day schedule, progress); and notifications/reminders (travel reminders, group updates, social-engagement alerts). Private post-trip collaborative hubs let groups compile photos and a shared scrapbook before itineraries are published to the community.

### Adjustments required for approval:

- **Clear the 2D gate with a custom-rendered data-display surface.** A native app built purely from a maps SDK plus native lists would collapse into ordinary CRUD and miss the module's 2D requirement. Custom-render at least one real 2D surface in **OpenGL ES (GLSurfaceView)** — e.g. the map

with your own POI/route/popularity-heat drawing and the day-by-day itinerary timeline, and/or the editor's live "rankings re-sort" view — rather than only a drop-in map widget and RecyclerViews.

- **Privacy & moderation for the social/publish layer.** Per-trip visibility (private hub vs friends vs published-to-community), report/block, and moderation of user photos/place data before public publish; don't expose precise personal/home locations. (Your private-hub model and per-trip control already point this way — make it an explicit, graded surface.)

*(Not required, affirmed:* the ranking/optimization/entity-resolution work is correctly scoped as the team's own load-bearing C++ engine — keep it that way. Map tiles/geocoding from a service are fine, but the scoring, re-ordering and dedupe stay in C++.)<sup>\*</sup>

#### *References:*

- [Wanderlog](#) (collaborative trip planner — create itineraries, discover, and plan trips together with friends)
- [Polarsteps](#) (plan, track & relive trips — itinerary builder, route on a world map, photos, sharing, and full per-trip privacy control)
- [Triplt](#) (all-in-one itinerary builder + trip planner with real-time flight alerts and travel reminders)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | Trippy |
|-------------------------------------------------------------|--------|
| External Database                                           | ✓      |
| Authentication »                                            | ✓      |
| Analytics »                                                 | ✓      |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓      |
| Application Communication                                   | ✓      |
| Continuous Integration                                      | ✓      |
| Data-Driven Architecture                                    | ✓      |
| Front-end + Back-end                                        | ✓      |
| Custom Tools Development                                    | ✓      |
| 2D Graphical Output                                         | ♦      |
| 3D (strong students only)                                   | ✕      |
| Networking System                                           | ✓      |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile — a native Android app (Kotlin UI + the C++ engine via JNI); the itinerary optimizer runs on-device (no network dependency) and the same C++ builds on the desktop for the editor/benchmarks (device: Pixel 9a). **Extended:** Firebase + Linux server set up by M2 (full use T2).

**Front end:** The map (custom-drawn POIs, routes, popularity), the day-by-day itinerary timeline and drag-to-reorder planner, discovery/search results, the collaborative group hub, the shared scrapbook and trip-progress views (OpenGL ES for the custom map/timeline surface). **Languages:** Web —

HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES) for the PC/desktop editor & benchmark build; Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

**Back end:** The C++ engine — itinerary scoring/ranking against constraints (friend-weighted), on-device stop/cluster re-ordering with per-step re-scoring, the spatial-metadata inference pipeline (dwell/seasonality/popularity), and proximity-based entity resolution — plus a Firebase store for spatial data, social graph, groups and media, and a notification service. **Languages:** C++.

**Backend scope:** Local now, server-ready (T2) — the optimizer runs on-device; the Linux server seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing piece is a **performant C++ service** — it scores hundreds of candidate itineraries against the traveller's taste, searches better orderings while re-scoring at each step, checks each new place against existing entries, and infers spatial metadata; it runs off the Kotlin UI thread over JNI and streams each improved itinerary to the screen. It has no Android/DB/UI dependency, so it also builds on the desktop for benchmarks and the regression suite.
- *Data-driven:* places and inferred metadata, itineraries, recommender weights, seasonality vectors, entity-resolution radii and the notification rules are hot-reloadable JSON — the editor retunes engine behaviour without recompiling.
- *2D:* the map (own POI/route/popularity rendering), the day-by-day itinerary timeline and the editor's live re-sort view are **custom-rendered in OpenGL ES** (see adjustment 1). *3D:* not needed. *Networking:* Firebase sync of the social graph/spatial data + push notifications (reminders, group updates, social-engagement alerts).
- *Custom tool:* the **C++ desktop editor** sharing the engine — add places, seed starter itineraries, retune recommender weights and watch rankings re-sort live, plus audio configuration.
- *External DB:* Firebase for spatial data, social graph, groups, media and logistics/documents. *Auth »:* accounts + friend graph. *Analytics »:* engagement events feed the spatial-metadata inference and stat views.

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** On the PC/desktop editor build, C++17/20 with a service loop that drives itinerary scoring and streams improved orderings; SDL2/3 dev harness, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G9 ranker and the on-device optimizer run through it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with places + inferred metadata, itineraries, recommender weights, seasonality vectors and entity-resolution radii in JSON (nlohmann/json) with hot-reload — the editor tunes engine behaviour against it.
- **Prerequisites:** G1(loop & update hooks).
- **Feeds into:** M1G9 (scoring reads it), M2G2 (the desktop editor edits it), M2G8 (search/optimizer features).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the ranker/optimizer, Tracy profiling (the search is CPU-bound), Catch2 regression + stress tests scoring/reordering large itinerary sets on the headless desktop build, GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).

- **Feeds into:** M2G3 (CI hardening) and the on-device latency budget M2G1 validates; the headless engine build is a named deliverable.
- **G9 Matching / Ranking Engine — Foundation** — the itinerary ranker/optimizer: score candidate itineraries against constraints and reorder stops.  
Tech & dependencies
  - **Tech:** Weighted scoring over itinerary features (pace, dates, preferences, friend-completion), local-search reordering of stops/day-clusters with per-step re-scoring, incremental re-rank (custom C++) — not a database query.
  - **Prerequisites:** G2 (place/itinerary model), G1 (tick).
  - **Feeds into:** M2G8 (intent search + on-device optimization II), the discovery feed.
- **G6 Custom 2D UI Rendering** — the map (POIs/routes/popularity) and the day-by-day itinerary timeline rendered custom (satisfies the 2D gate).  
Tech & dependencies
  - **Tech: Web build:** SDL2/3 renderer or OpenGL ES, GLSL, FreeType, stb\_image for custom map markers/routes/popularity heat and the itinerary timeline on the desktop build. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** G1 (shell & loop).
  - **Feeds into:** The planner/discovery views; the editor's live re-sort visualization.
- **G8 Firebase Integration — Identity & Data** — accounts, the social graph/groups, and the crowdsourced spatial dataset & media.  
Tech & dependencies
  - **Tech:** Firebase Auth, Firestore/Realtime DB for places/itineraries/groups/media, REST API from C++ and the Kotlin SDK on the front-end.
  - **Prerequisites:** G2 (data model to sync).
  - **Feeds into:** M2G6 (entity-resolution/moderation), M2G12 (notifications), friend-weighted ranking.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

#### Tech & dependencies

- **Tech: Web build:** Tracy profiling of the search/optimizer, ASan on MSVC, memory pools, and validation on the desktop editor/benchmark build; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI; validate on-device optimizer latency and the GLSurfaceView map/timeline on the Pixel 9a.
  - **Prerequisites:** All M1 goals (hardens the M1 codebase).
  - **Feeds into:** The native Android deliverable and instant on-device re-planning.
- **G2 ★ Content Editor — Core** — the C++ desktop editor sharing the engine: add places, seed starter itineraries, retune recommender weights and watch rankings re-sort, with audio config.

#### Tech & dependencies

- **Tech:** C++ + Dear ImGui, JSON serialization, undo/redo command stack; live binding so a weight change re-sorts the visible rankings, plus map-based place authoring.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live map + re-sort preview).
- **Prerequisites:** G2 (data model), M1G9 (engine to drive/inspect).
- **Feeds into:** Seeded starter content; weight tuning that M2G8 consumes.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

#### Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the desktop editor/benchmark build via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI) packaging the Android APK/AAB; OpenGL ES in a GLSurfaceView where realtime 2D is needed — no SDL on native.
  - **Prerequisites:** G3 (CI and test foundation).
  - **Feeds into:** Release/showcase builds of every Trippy goal.
- **G8 Ranking / Matching Engine II** — intent-based itinerary search + on-device optimization II: friend-weighted ranking and instant re-planning to a new user's pace/schedule/season.

#### Tech & dependencies

- **Tech:** Scoring models (weighted features, similarity), candidate generation, evaluation metrics, and a local-search optimizer that streams improving itineraries; social-connection completion as a ranking signal.
  - **Prerequisites:** M1G9 (ranking foundation), M1G8 (social/data).
  - **Feeds into:** The intent search, personalised recommendations and instant re-plan.
- **G12 Notification II** — travel reminders, real-time group updates and social-engagement alerts.

#### Tech & dependencies

- **Tech:** Firebase Cloud Messaging, digest/schedule for reminders (flights, check-ins, activities), and push on group modifications and scrapbook/stat/trip-log posts.
  - **Prerequisites:** M1G8 (Firebase identity/data).
  - **Feeds into:** Retention; the group collaboration loop.
- **G6 Content Moderation Dashboard & Pipeline** — crowd-validated entity resolution + publish-time moderation.

#### Tech & dependencies

- **Tech:** Proximity-based duplicate detection and a human-in-the-loop consensus flow to a canonical place; a report/review queue and moderation dashboard for user photos/place data before public publish; Firebase security rules.
- **Prerequisites:** M1G2 (place model), M1G8 (Firebase identity/data).
- **Feeds into:** A precise, trusted spatial database; safe community publishing.

*(Proposed new pool goal (requires instructor approval):* **Spatial Metadata Inference Pipeline** — infer dwell time, pacing, seasonality vectors and spot popularity from recorded journeys (aggregation + custom charting); the Social Media pool's nearest is the Activity Analytics Engine, but the inference angle is novel. Fold into M1G2/M2G8 or append as a new pool goal; the three picks above stand regardless.)

## Track Goals & Team Composition

**Recommended composition:** 6 CS + 2 Design + 0 Art (8) — Trippy is a UX-heavy social utility: collaborative planning, friend-sourced discovery, day-by-day tracking, the group hub/scrapbook and notifications are a strong two-designer load, while the visual surface (map markers, badges, icons, scrapbook layouts) is UI-art the designers own and the photos are user-supplied — no character/illustration production, so 0 artists (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see [team-formation.pdf](#)). With 0 artists, the designers take the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality) — the last Design goal each milestone.

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the plan → discover → collaborate → track → publish loop, the friend-sourced discovery model, and the private-hub → community-publish flow)
- M1D2 Core-Loop Interactive Prototype (Figma flow: create trip → discover/rank → plan-with-friends → day-by-day → publish)
- M1D3 User Research, Personas & Competitive Analysis (teardown of Wanderlog / Polarsteps / Triplt; traveller personas)
- M1D5 Interaction & Input Design (map interactions, drag-to-reorder itinerary, group collaboration, proximity-prompt entity resolution)
- M1D11 Art Proof of Concept (conditional — no artist: map-marker/badge/icon set, scrapbook visual language, UI style guide)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D2 Visual Branding & Style Guide (Trippy identity; the trip-stat / badge system)
- M2D5 Playtesting & Iteration Program (planning friction, discovery relevance, collaboration clarity)
- M2D11 Art Implementation Quality (conditional — no artist: UI art/icons implemented to standard)

- **UI/UX M1** (graded by the UI/UX instructor):
  - M1U1 ★ UI Standards & Wireframe Baseline — the map, planner, discovery, group hub and tracking screens.
  - M1U2 UX Flows & Wireframes — create → discover → plan-with-friends → track → publish journeys.
  - M1U3 Accessibility & Readability Foundations — legible timeline/ map typography, contrast, thumb-reach targets.
- **UI/UX M2:**
  - M2U1 ★ Functional UI & Usability — the required screens navigable unaided on the mobile target.
  - M2U2 ★ UX Validation & Final Polished UI — validated, polished planner, map and group hub.
  - M2U3 Menu & HUD Motion Design — itinerary re-sort animation, map transitions, notification affordances.
  - M2U5 Mobile UX Adaptation & Ergonomics — one-handed planning and map use on the Pixel 9a, offline-friendly.
- **Art:** 0 artists — the art surface is covered by the D11 conditionals, taken as Design goals (see Design M1/M2).
- **TEAM M1:**
  - M1TA1 ★ Overall Audio Experience — light UI/notification SFX and optional ambience from the week-5 needs list, present & working in the prototype (added stop, reminder, publish), suitable and correctly triggered. Audio is not load-bearing here — a lecturer scope waiver may apply (the editor's audio-config hook aside).
  - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping UI SFX.
- **TEAM M2:**
  - M2TA1 ★ Overall Audio Experience — the standard floor tailored down for a utility app (subtle UI/notification SFX + optional ambience in the showcase build).
  - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing for stacked notifications; the desktop editor's audio-config hook).

- M2TP1 ★ Final Presentation — 7 min in front of class +  
instructors: live custom-tools showcase (the C++ desktop editor),  
prototype demo, team post-mortem (wk14)
- 

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*